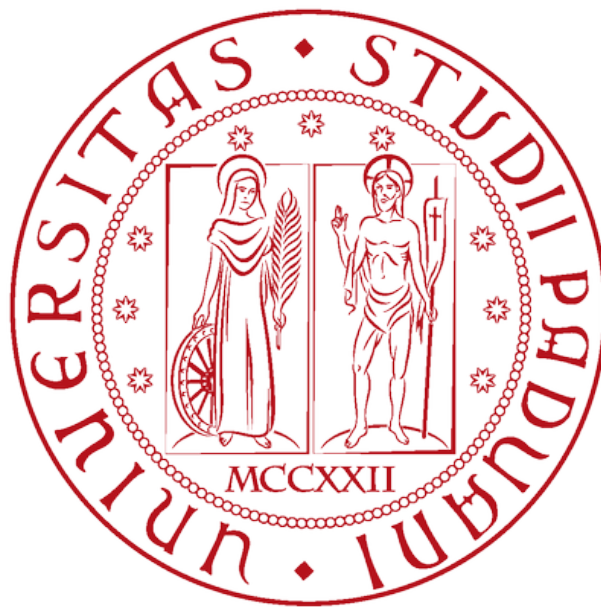


UNIVERSITY OF PADOVA

Embedded Real-Time Control

Laboratory report



Names - (Student ID Numbers)

Academic Year 2021-2022

Contents

1	General notes	2
2	Laboratory 1	3
2.1	SX1509 I/O expander	3
2.2	Line sensor	4
2.3	Keypad	4
2.4	Laboratory work	5
2.5	Conclusion	6
3	Laboratory 2	6
3.1	Timer Configuration	6
3.2	IMU Data Handling	7
3.3	Servo Control	7
3.4	Data Logging	7
3.5	Results	8
4	Laboratory 3	9
4.1	Design process	9
5	Laboratory 4	9
5.1	Design process	9
5.2	Error definition	9
5.3	Yaw controller Design	10
5.4	Design process	12
A	Section in the Appendix	13
A.1	Subsection in the Appendix	13

1 General notes

This report was made as part of evaluation of course Embedded Real-Time Control at UNIPD. In labs we were using and all development was done on device named TurtleBot (see picture 1). As software development tool was used STM32 CUBE IDE.

We are using microcontroller from STM32F7 series, part of STMicroelectronics' STM32 family, is based on the ARM Cortex-M7 core, operating at frequencies up to 216 MHz. It is designed for high-performance embedded applications that require substantial processing power combined with real-time capabilities. The STM32F7 features up to 2 MB of dual-bank Flash memory, up to 512 KB of SRAM (with additional TCM memory), and includes an integrated Floating Point Unit (FPU) and Digital Signal Processing (DSP) instructions for efficient signal processing tasks. Key peripherals include high-speed USB OTG (with support for full-speed and high-speed modes), multiple 12-bit ADCs and DACs, timers with encoder and PWM modes, as well as extensive connectivity options such as UART, SPI, I²C, CAN, SDIO, and Ethernet MAC with IEEE 1588 support. The series also includes an L1 cache (I-cache and D-cache), Chrom-ART Accelerator for graphical applications, and flexible memory interfaces for external SDRAM, SRAM, or NOR/NAND Flash. These features make the STM32F7 well-suited for applications involving advanced user interfaces, audio processing, motor control, and industrial automation.



Figure 1: The TurtleBot

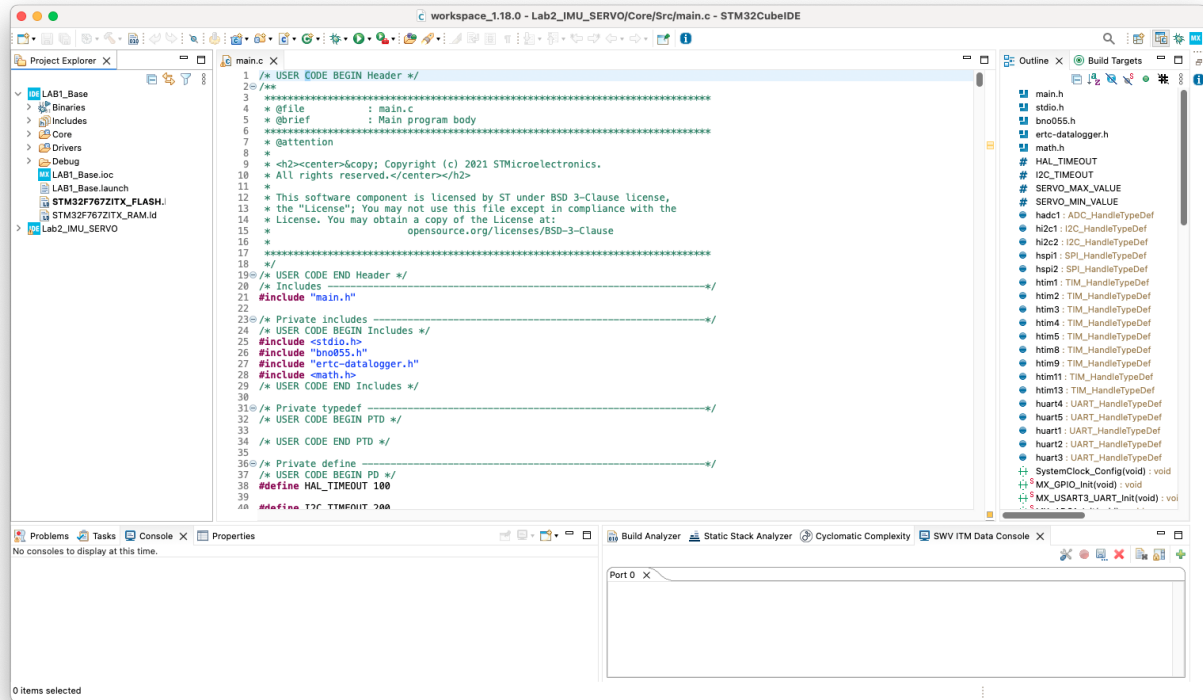


Figure 2: example from STM32 cube IDE

2 Laboratory 1

Target of the first lab was to establish I2C communication between mcu STM32 and I/O expander SX1509. And to read the data from the keypad and the line sensor with help of this expander. See configuration on picture 3.

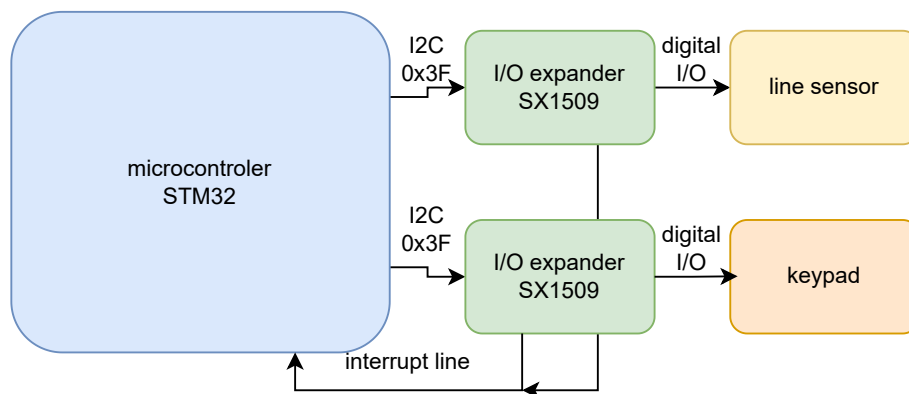


Figure 3: The TurtleBot

2.1 SX1509 I/O expander

We are using 16-ch digital I/O expander SX1509 [1]. Device has two I/O banks, represented by two registers in device memory of address 0x27 and 0x28. The I2C address of device is configurable by 2 address pins. Up to 4 devices could be at one line. We are using addresses 0x3E and 0x3F since we have 2 devices. Chip has built in debouncing, keypad engine and LED driver. See scheme on the picture 4

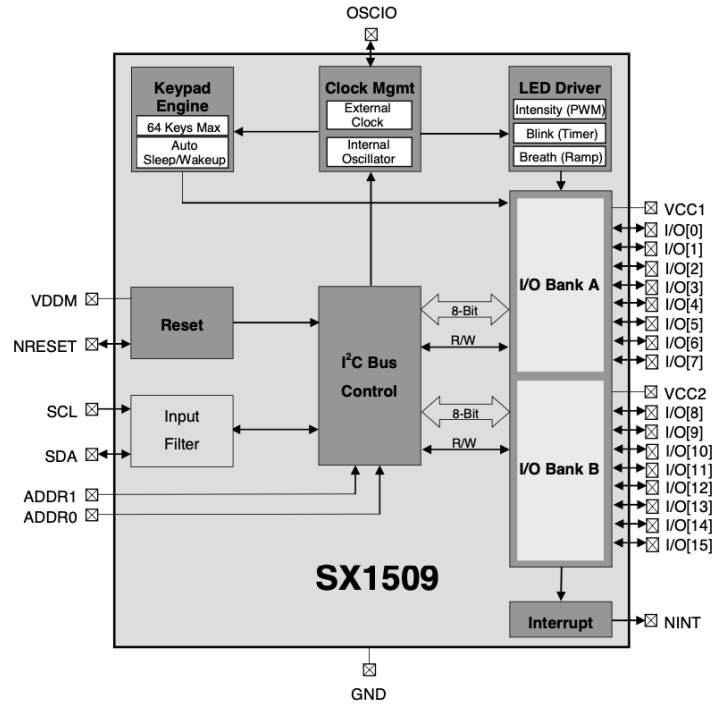


Figure 4: SX1509 block scheme

2.2 Line sensor

TurtleBot has Pololu QTR sensor[2] onboard, version with 8 channels and analog output. Each one of the outputs is connected to one of the digital inputs of SX1509. Internal logic of expander converts signal to digital one when is voltage higher/lower than treshhold. Order of bits coresponds to physical state of the sensor. Principle of sensor is shown on picture 5.

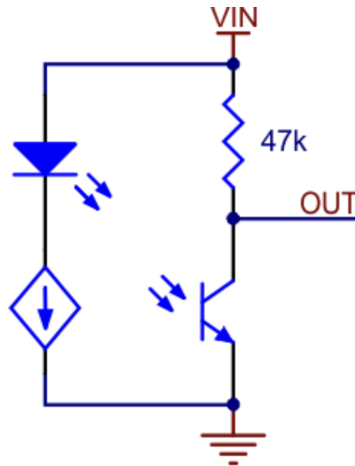


Figure 5: Scheme of single unit from line sensor

2.3 Keypad

Classical 4x4 matrix keypad is used. Digital signal is connected to I/O expander. Rows are connected to the one bank (0x28) and columns (0x27) to the other one, so four low bits of represents the keys. See keyboard configuration on picture 6.

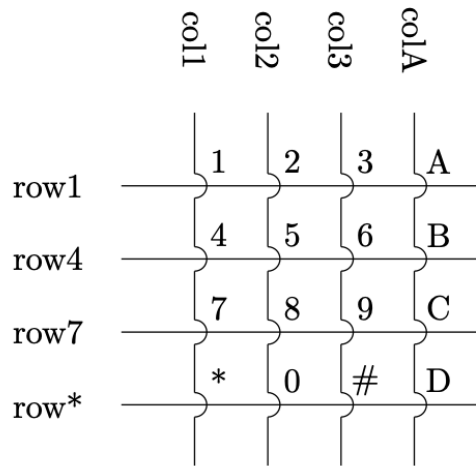


Figure 6: Scheme of keypad connection

2.4 Laboratory work

Exercise 1 & Exercise 2: Write an ISR that recognizes and correctly handles the keypad interrupts. You have to properly implement the void HAL_GPIO_EXTI_Callback(uint16_t pin) function. Then print which interrupt has been triggered, i.e., the pin. To be able to receive another interrupt from the keypad, have to read the registers REG_KEY_DATA_1 and REG_KEY_DATA_2 inside the ISR.

Extend the code of exercise 1 to handle the keypad interrupt. You have to print which keypad button has been pressed.

We have implemented one callback function, for reading keypad data only when the button is pressed. We are returning 2 global variables. One for interrupt pin number and one as flag, so we can execute appropriate part of code in main. Then we calling function to read data from row and columns through I2C. See listing 1.

Listing 1: Keypad callback function

```

1  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2  {
3      IRQ_GPIO_pin = GPIO_Pin;
4      IRQ_GPIO_flag = 1;
5      HAL_I2C_Mem_Read(&hi2c1, I2C_KEYPAD_ADDRESS, KEYPAD_ROWS_ADDRESS,
6      8, key_rows_data, 1, I2C_TIMEOUT);
7      HAL_I2C_Mem_Read(&hi2c1, I2C_KEYPAD_ADDRESS, KEYPAD_COLS_ADDRESS,
8      8, key_cols_data, 1, I2C_TIMEOUT);
9  }
```

To get character from the keypad, we need to decode the data we received from the expander. For that purpose we have two functions, you can see in listing 2.

Listing 2: Keypad get cahr functions

```

1  uint8_t GetKeypadPosition(uint8_t data)
2  {
3      if(data == 254)
4          return 0;
5      else if(data == 253)
6          return 1;
7      else if(data == 251)
8          return 2;
9      else
10         return 3;
11 }
12
13
```

```

14     char GetCharFromKeypad( uint8_t const rows, uint8_t const cols )
15     {
16         return keypadLayout[rows][cols];
17     }

```

Exercise 3: Write a routine that reads the status of the line sensor and prints it. The routine must check the status with a polling period of 100ms.

We implemented function for reading data form the light sensor (see 4), and we calling this function by polling in main every 100 ms. We basically just read device registers trough I2C.

Listing 3: Keypad get cahr functions

```

1     void GetLightSensorStatus( uint8_t * data )
2     {
3     HAL_I2C_Mem_Read(&hi2c1, I2C_LIGHT_SENSOR_ADDRESS, LINE_SENSOR_ADDRESS,
4     1, data, 1, I2C_TIMEOUT);
5     }

```

Exercise 4: We haven't saved LAB0, since it is not part final evaluation, so we can extend it, but since our LAB0 was working properly, it is basically just changing LED delay based on keypad input. That means making a function that returns delay value based on input combination.

2.5 Conclusion

In this lab we establish I2C communication with two devices and successfully read data from these. We implemented keypad decoder, successfully read data from line sensor and haven't finished exercise 4, because we haven't saved lab0 on flash disk and were not able to login to school comuputer for 20 minutes. At least we make theoretical description of the problem, since it is quite trivial. Some of the functions are short so it would make sense to make them inline or not using function at all. We are not aiming for the most effective code and we prefer readability. Our code was not compiling with inline versions so we are using regular functions.

3 Laboratory 2

The aim of this laboratory was to implement an open-loop camera stabilization system on the TurtleBot using data from its IMU and controlling two servo motors to adjust the tilt and the pan of the camera.

3.1 Timer Configuration

In order to get a 100Hz update frequency, we have the following timer configuration in place:

1. **Prescaler:** The prescaler is used to divide the input clock frequency of 96 MHz by 9600 in order to set the timer's counting frequency to 10kHz;
2. **Counter Period:** With the counter period of 100, we thus get the desired update frequency of $10kHz/101 \approx 100Hz$.

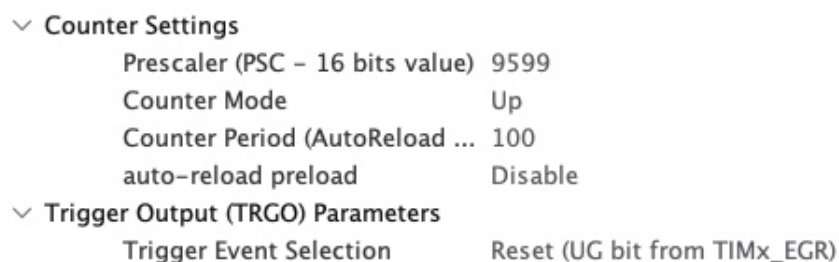


Figure 7: TIM6 Configuration.

3.2 IMU Data Handling

The TurtleBot is equipped with a 6-DOF IMU, comprised of a 3-axis accelerometer and 3-axis gyroscope. The accelerometer measures the linear acceleration (m/s^2) along the 3 axes, while the gyroscope measures angular velocity (rad/s) around these axes. The data from the two sensors is available within

`bno055_convert_double_accel_xyz_msq()` and `bno055_convert_double_gyro_xyz_rps()`, respectively. The data provided by the IMU is crucial when estimating the movement and orientation of the robot, particularly for calculating the tilt as follows:

$$\theta = \sin^{-1}\left(\frac{a_y}{g}\right)$$

where a_y is the acceleration along the y axis.

3.3 Servo Control

The two servo motors controlling the movement are operated via PWM signals generated by TIM6 of the STM32 and the angles are updated with a frequency of 100Hz using the `__HAL_TIM_PeriodElapsedCallback` function and scaled into a safe duty cycle with the `saturate()` function.

Listing 4: Angle update function

```
1 TIM_HandleTypeDef * timer6_ptr = &htim6;  
2 HAL_TIM_PeriodElapsedCallback(timer6_ptr)  
3 {  
4     theta = asin(d_accel_xyz.y/G);  
5     pan = (int8_t)(-(theta/3.14)*180) + PAN_BIAS;  
6  
7     angle = angle + ((-(d_gyro_xyz.z*0.1)/3.14)*180.)/10.;  
8  
9     tilt = (int8_t)angle;  
10 }
```

As can be seen in the code, there is a strange phenomenon where dividing the angle update formula by 10 ensures a smooth pan without the need of a filter. While this makes no sense time/frequency-wise (after having done the calculations multiple times), it does in the end provide a very smooth movement. The most probable reason is that, by reducing the amplitude of the response, some noise from the gyroscope is actually "filtered".

3.4 Data Logging

Data logging was performed via the UART interface of the STM32, while using MATLAB to receive and visualize the data in real time with the `serial_data_log()` function. The plots for the yaw compensation and for the tilt compensation can be seen in Figure 8 and Figure 10, respectively.

The data from graphs 8 and 10 was obtained by moving the TurtleBot according to the Figure 9(data for the yaw movement) and Figure 11(data for the tilt movement), respectively.

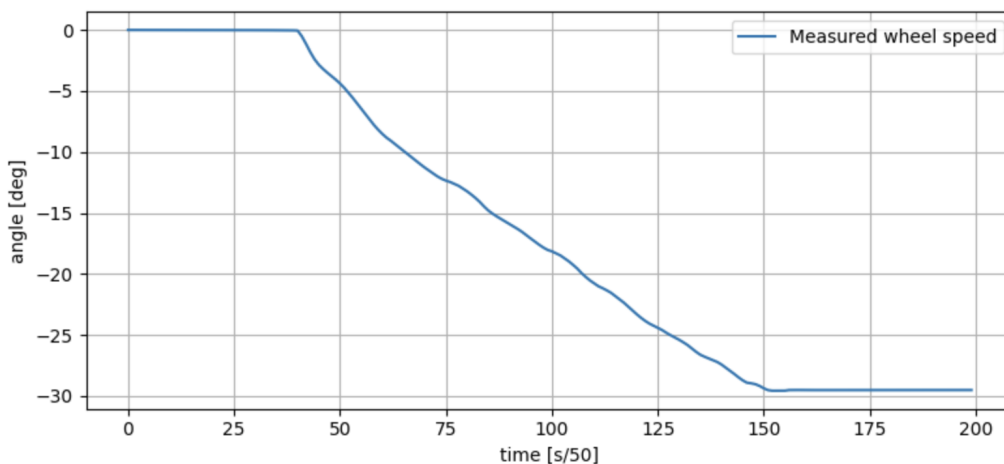


Figure 8: Yaw response behaviour.

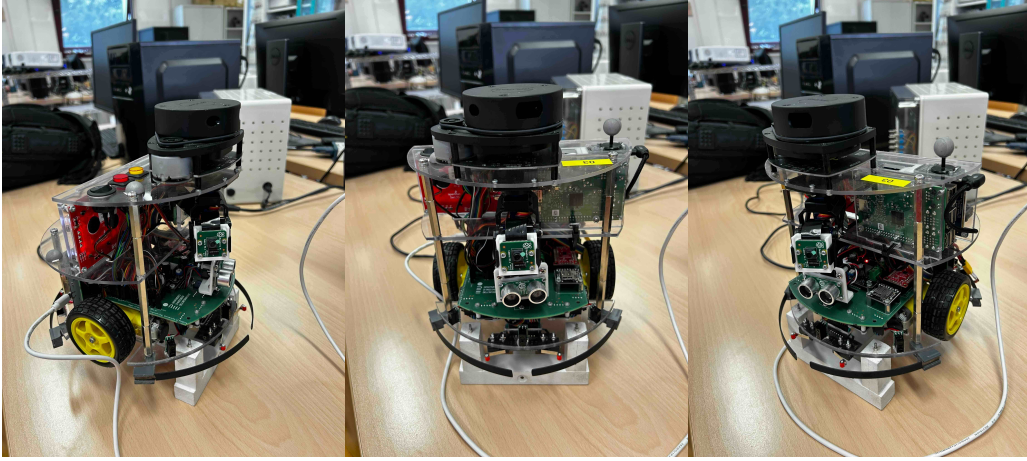


Figure 9: Yaw response behaviour.

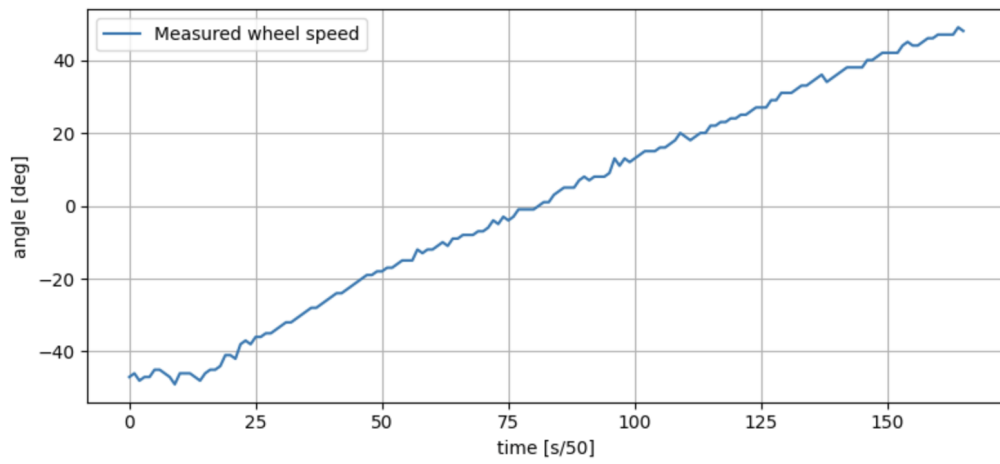


Figure 10: Tilt response behaviour.

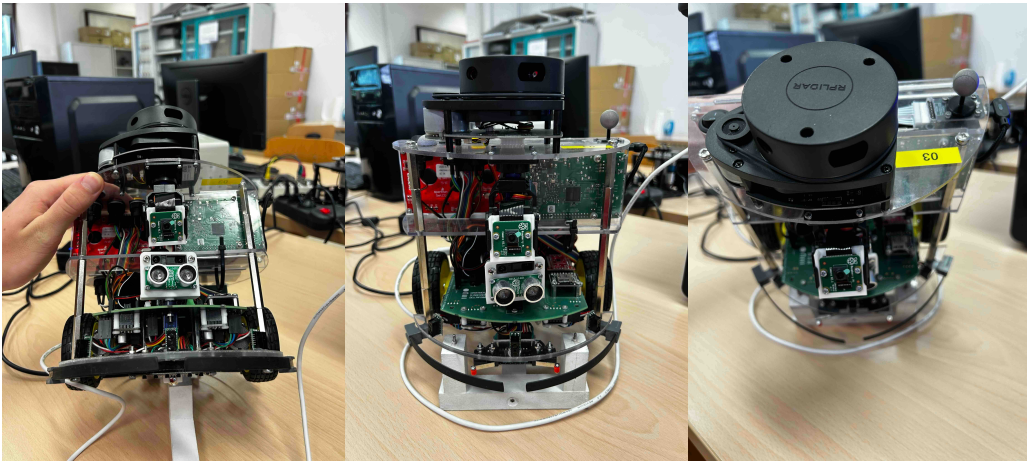


Figure 11: Tilt response behaviour.

3.5 Results

Lab video demonstration::Video Demonstration

The camera stabilization performance was captured and logged while the TurtleBot was tilted manually. As seen in the video, the servo effectively counteracts the tilt to maintain camera alignment, without the need of a

complementary filter. In the event of a less fortunate camera setup, the complementary filter would have had the following form:

$$\theta_{fused} = \alpha \cdot \theta_{gyro} + (1 - \alpha)\theta_{accel} \quad (1)$$

, where θ_{gyro} and θ_{accel} are two different sensor measurements, from the gyroscope and accelerometer, respectively, thus ensuring a smoother pan.

4 Laboratory 3

4.1 Design process

We implemented the motor controller relatively quickly and tested it. Initially, everything seemed to work fine, so we proceeded with final design adjustments. Unfortunately, we introduced a small change that caused the error to be computed with the wrong sign. This mistake cost us a significant amount of time, as we assumed the implementation was correct — and since it worked intermittently, it was even more misleading. We also implemented the jaw controller but struggled with tuning it properly. Eventually, we discovered the source of the issue. Unfortunately, we ran out of time during the lab session and were not able to tune the PID controller properly.

5 Laboratory 4

5.1 Design process

At first, we tried to implement a simple approach, but it did not work as we expected. The idea was to split the left and right side of the sensor into two separate arrays, giving us two values. Each value represented the occupancy of one half of the sensor. We then subtracted these values to get an error — ideally close to zero.

In hindsight, our implementation may not have been flawed, the real issue was likely the motor control implementation. At the time, however, we assumed the problem was in our logic. We implemented the jaw controller, without major issues. It worked as expected.

5.2 Error definition

For starters, let us define our yaw error:

$$\psi_{err} = \psi_{ref} - \psi \quad (2)$$

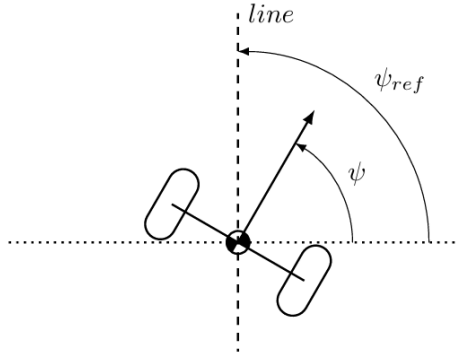


Figure 12: Yaw error definition

As shown in the picture, the error that the yaw controller is receiving is made from the difference between the reference and the current error. We want that the error to be minimal. It is also possible to rewrite the equation stated above with respect to the error of following the line, namely e_{esl} .

$$\psi_{err} = \arctan\left(\frac{e_{esl}}{H}\right) \quad (3)$$

, where H is the distance between the line sensor and the vehicle's center, equal to 0.085 m and e_{esl} is defined as:

$$e_{esl} = \begin{cases} 0, & \text{when the line is at the center} \\ > 0, & \text{when the line is on the left side of the sensor} \\ < 0, & \text{when the line is on the right side of the sensor} \end{cases} \quad (4)$$

Therefore, using basic mathematical knowledge, the $\arctan(x)$ is not needed when the value of x is close to 0 (zero). In conclusion, we can define the yaw error as follows:

$$\psi_{err} = \frac{e_{esl}}{H}; \quad (5)$$

We implemented the yaw error by as follows:

Listing 5: Yaw error

```

1
2  turtle_bot->TB_jaw_error = (line_attributes->distance_from_center/LINE_SENSOR_DISTANCE)

```

5.3 Yaw controller Design

For the design of the Yaw controller we implemented the following structure in Matlab Simulink:

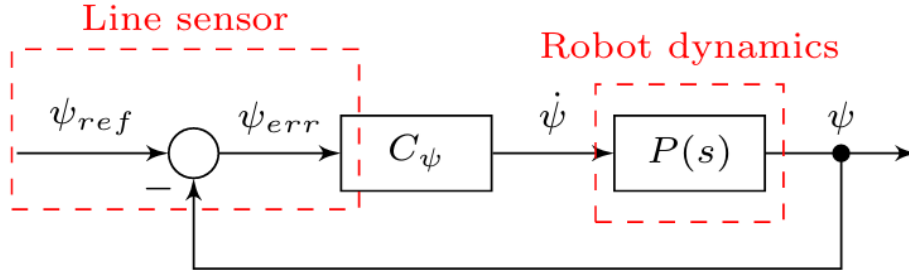


Figure 13: Yaw controller

For the controller, namely C_ψ , is multiplied by the error that we have defined above. Taken this into consideration, we will define the controller of the yaw by implementing a proportional (P) control. The value of the yaw controller is defined by the variable K_r .

Listing 6: Yaw controller

```

1
2  pid_attributes->output = pid_attributes->Kr*(TB->TB_jaw_error);

```

With everythind defined, we can introduce the Simulink model implemented by us:

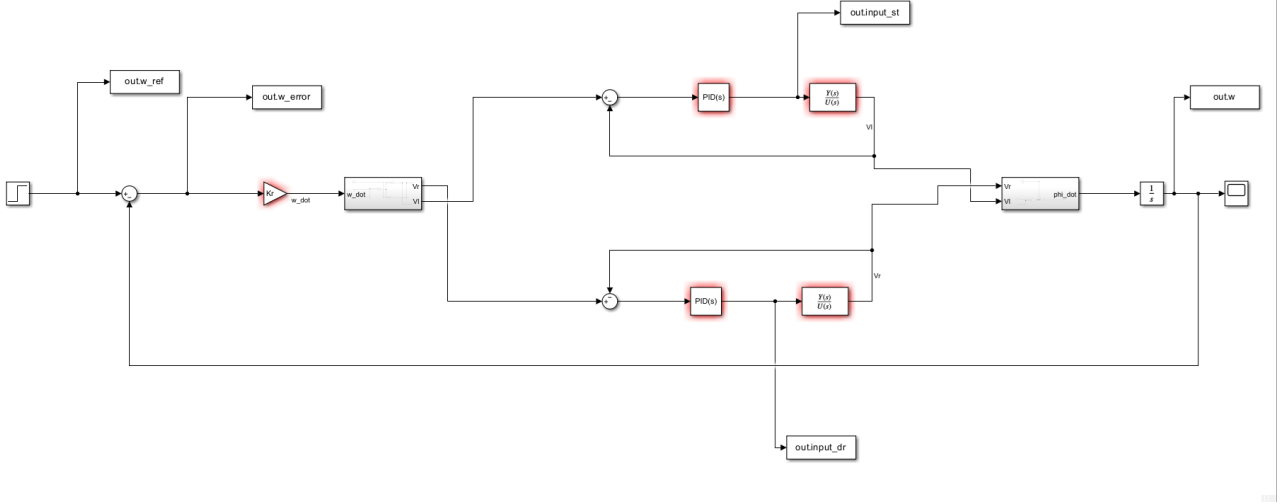


Figure 14: Simulink model

We can clearly see both the PID-s that are intended for controlling both of the wheels speeds. In the efforts of finding the suitable PID for both of the wheels such that the overall error is minimal, I implemented a metaheuristic method using the Matlab implemented function, namely PSO. PSO stands for Particle Swarm Optimization and it is a method in which we can find the parameters of both the PID-s and the yaw controller gain by running the simulation multiple times and placing particles all over the search space.

The particles have unique attributes, such as :

1. they have knowledge of their position in the search space
2. they have knowledge of the swarms position in the search space
3. they have knowledge of their value
4. they have knowledge of the swarms values

The goal of this metaheuristic is to find the global optimal value that satisfies the condition given by the user. The condition given is that we want the overall error to be minimal. This can be described as :

$$J = \|\psi_{error}\|; \quad (6)$$

In this way, the goal of the algorithm is to move the particles in the search space at every iteration, then the particles, by running the simulation with the values obtained in that iteration, obtain an output that is being normalized for easier comparison. In the end, after multiple iterations, the particles should gather in a small area which in the end is the global maximum value, namely the minimal error found in the designated search space.

For the algorithm to work properly, I have implemented different intervals that the values should take. Therefore, we can run the same algorithm, but with multiple choices, building multiple configurations.

The preset configurations are :

1. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; ai, (ai + bi)/2; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
2. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; ai, (ai + bi)/2; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$
3. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; (ai + bi)/2, bi; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
4. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; (ai + bi)/2, bi; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$
5. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; ai, (ai + bi)/2; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
6. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; ai, (ai + bi)/2; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$
7. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; (ai + bi)/2, bi; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
8. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; (ai + bi)/2, bi; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$

a_p and b_p are the intervals for the Proportional gain, a_i and b_i are the intervals for the Integral gain, a_d and b_d are the intervals for the Derivative gain and a_r and b_r are the intervals for the Yaw controller gain.

After simulating all the configurations, we have come up with these values:

1. $K_{left}^p = 5,46$
2. $K_{right}^p = 5.193$
3. $K_{left}^i = 51.276$
4. $K_{right}^i = 47.853$
5. $K_{left}^d = 0.505$
6. $K_{right}^d = 0.488$
7. $K_r = 1095$

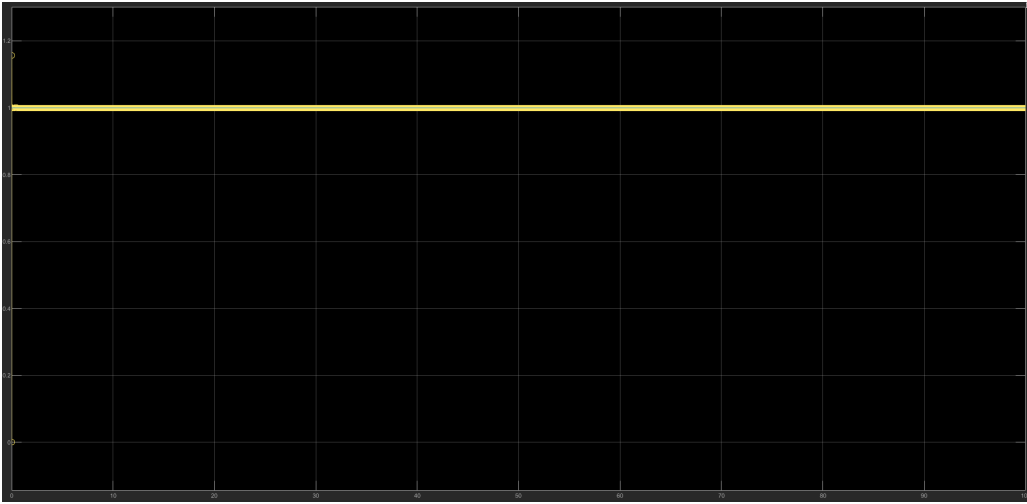


Figure 15: Output with Input

As we can see, the output perfectly tracks the the step reference with these values. Taking this into account, we call it a success in tracking the step reference. Although, by testing these values on the bot, we could not get the results that we were hoping for. I think this is to the fact that the reference is different or the sensor does not read as the simulation intends.

5.4 Design process

At first, we tried to implement a simple approach, but it did not work as we expected. The idea was to split the left and right side of the sensor into two separate arrays, giving us two values. Each value represented the occupancy of one half of the sensor. We then subtracted these values to get an error — ideally close to zero.

In hindsight, our implementation may not have been flawed, the real issue was likely the motor control implementation. At the time, however, we assumed the problem was in our logic. We implemented the jaw controller, without major issues. It worked as expected.

A Section in the Appendix

A.1 Subsection in the Appendix

Additional relevant information...

References

- [1] SX1507/SX1508/SX1509. 2025. *Sparkfun* [online]. [accessed 2025-4-22]. Available at:
<https://cdn.sparkfun.com/datasheets/BreakoutBoards/sx1509.pdf>
- [2] Pololu line sensor. 2025. *Pololu.com* [online]. [accessed 2025-4-22]. Available at:
<https://www.pololu.com/docs/pdf/0J12/QTR-8x.pdf>